

UNIVERSIDAD DEL VALLE DE GUATEMALA

Colegio Universitario



Proyecto 1 – Uso de un protocolo existente

Model Context Protocol (MCP)

Camila Richter 23183

CC3067 – Redes

8 de septiembre del 2026
Ciudad de Guatemala, Guatemala

1. Introducción

Los Large Language Models (LLM) poseen la capacidad de interpretar lenguaje natural, generar contenido y mantener el contexto de una conversación. Sin embargo, no pueden interactuar directamente con recursos externos, como sistemas de archivos, bases de datos o servicios disponibles a través de Internet.

Model Context Protocol (MCP) busca resolver este problema con un protocolo abierto que estandariza la manera en que aplicaciones basadas en inteligencia artificial descubren y usan herramientas externas. MCP establece una separación entre el anfitrión que contiene la aplicación de inteligencia artificial, los clientes que mantienen las conexiones y los servidores que exponen las herramientas.

El objetivo de este proyecto fue implementar un chatbot capaz de comunicarse con un LLM mediante su API y usar diferentes servidores MCP locales y remotos. También se desarrolló un servidor MCP propio, Academic Planner MCP, orientado a la planificación académica universitaria, y un segundo servidor MCP remoto.

2. Arquitectura general de la solución

La solución desarrollada utiliza una arquitectura compuesta por un chatbot anfitrión, un LLM y múltiples servidores MCP.

El usuario interactúa con el chatbot desde una interfaz de línea de comandos. El chatbot mantiene el contexto de la conversación y se comunica con el LLM usando la API de Anthropic. Funciona como anfitrión de diferentes clientes MCP, encargados de mantener las conexiones con los servidores disponibles.

Los servidores utilizados durante el proyecto fueron:

- Filesystem MCP Server.
- Git MCP Server.
- Academic Planner MCP Server.
- Dos servidores MCP desarrollados por otros estudiantes.
- Academic Remote MCP Server.

El chatbot es responsable de proporcionar al modelo las herramientas. Cuando el LLM determina que una consulta requiere una de estas herramientas, el chatbot identifica el servidor MCP correspondiente, realiza la llamada y envía el resultado nuevamente al modelo para que genere la respuesta.

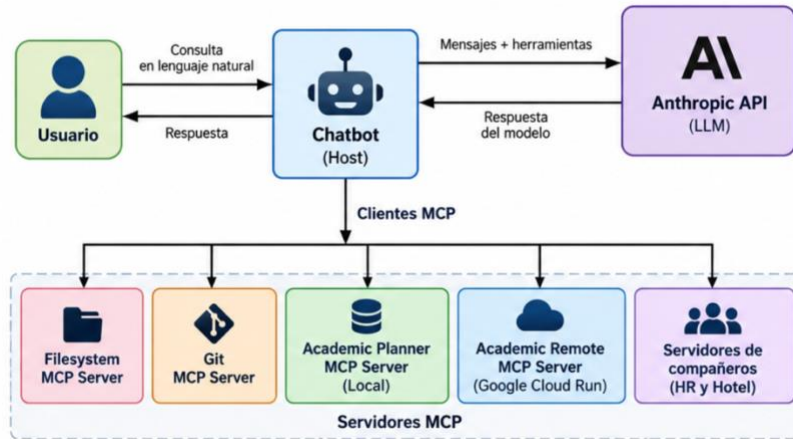


Figura 1. Arquitectura general del chatbot y servidores MCP utilizados.

3. Implementación del chatbot anfitrión

3.1 Conexión con el LLM

El chatbot fue desarrollado en Python y usa la API de Anthropic para comunicarse con el LLM.

La API permite enviar el historial de mensajes de la conversación con la descripción de las herramientas disponibles. Así, el modelo puede realizar dos tipos de acciones, generar una respuesta basada en su conocimiento o solicitar la ejecución de una herramienta externa.

Esta separación es importante debido a que el LLM no ejecuta directamente las operaciones. El modelo solo determina qué herramienta necesita y cuáles parámetros deben utilizarse. La ejecución efectiva corresponde al servidor MCP.

```

○ (venv) Camila@Mac academic-mcp-chatbot % python src/main.py

=====
Academic AI Assistant
=====
Secure MCP Filesystem Server running on stdio
Client does not support MCP Roots, using allowed directories set from server args: [ '/Users/Camila/mcp-sandbox' ]

[MCP] Tools disponibles: ['read_file', 'read_text_file', 'read_media_file', 'read_multiple_files', 'write_file', 'edit_file',
'create_directory', 'list_directory', 'list_directory_with_sizes', 'directory_tree', 'move_file', 'search_files', 'get_file_in
fo', 'list_allowed_directories', 'git_status', 'git_diff_unstaged', 'git_diff_staged', 'git_diff', 'git_commit', 'git_add', 'g
it_reset', 'git_log', 'git_create_branch', 'git_checkout', 'git_show', 'git_branch', 'add_academic_task', 'get_upcoming_tasks'
, 'update_task_status', 'calculate_task_priority', 'analyze_workload', 'generate_study_schedule', 'recommend_study_technique'
, 'decompose_project', 'get_random_study_tip', 'get_employee', 'search_employees', 'get_vacation_balance', 'calculate_overtime
pay', 'get_payroll_summary', 'get_employee_history']

You > ¿Quién es el creador de Facebook?

Assistant > Mark Zuckerberg es el creador de Facebook. Junto con sus compañeros de la Universidad de Harvard (Eduardo Saverin,
Andrew McCollum, Dustin Moskovitz y Chris Hughes), fundó la plataforma en 2004.

```

Figura 2. Comunicación directa entre el chatbot y el LLM para una consulta general.

3.2 Mantenimiento del contexto

El chatbot mantiene el historial de mensajes enviados durante una sesión. Cada nueva interacción incorpora el contexto de los mensajes anteriores.

Por ejemplo, una interacción puede iniciar con una pregunta acerca de una persona y posteriormente realizar una segunda pregunta utilizando únicamente un pronombre o una referencia implícita. Debido a que el historial se mantiene, el modelo puede determinar a qué entidad corresponde la segunda consulta.

El mismo mecanismo resulta importante para Academic Planner, ya que permite mantener conversaciones donde el usuario agrega información progresivamente. La última consulta puede ser interpretada usando el contexto de las interacciones anteriores.

```
○ (venv) Camila@Mac academic-mcp-chatbot % python src/main.py

=====
Academic AI Assistant
=====
Secure MCP Filesystem Server running on stdio
Client does not support MCP Roots, using allowed directories set from server args: [ '/Users/Camila/mcp-sandbox' ]

[MCP] Tools disponibles: ['read_file', 'read_text_file', 'read_media_file', 'read_multiple_files', 'write_file', 'edit_file',
'create_directory', 'list_directory', 'list_directory_with_sizes', 'directory_tree', 'move_file', 'search_files', 'get_file_in
fo', 'list_allowed_directories', 'git_status', 'git_diff_unstaged', 'git_diff_staged', 'git_diff', 'git_commit', 'git_add', 'g
it_reset', 'git_log', 'git_create_branch', 'git_checkout', 'git_show', 'git_branch', 'add_academic_task', 'get_upcoming_tasks',
'update_task_status', 'calculate_task_priority', 'analyze_workload', 'generate_study_schedule', 'recommend_study_technique',
'decompose_project', 'get_random_study_tip', 'get_employee', 'search_employees', 'get_vacation_balance', 'calculate_overtime
pay', 'get_payroll_summary', 'get_employee_history']

You > quién fundó apple?

Assistant > Apple Inc. fue fundada en 1976 por Steve Jobs, Steve Wozniak y Ronald Wayne.

Un breve resumen de cada uno:

- Steve Jobs: Fue el visionario y líder detrás del desarrollo de productos y la estrategia de la compañía.
- Steve Wozniak: Ingeniero que diseñó el Apple I y el Apple II, las primeras computadoras de la empresa.
- Ronald Wayne: Colaboró en la redacción del contrato de sociedad inicial, pero vendió su participación (un 10%) apenas 12
días después de la fundación, por lo que es menos conocido.

Cabe mencionar que mi función principal es ayudarte con tareas relacionadas a gestión académica (tareas, exámenes, horarios de
estudio) o de recursos humanos (nómina, vacaciones, empleados). Si tienes alguna consulta sobre esos temas, ¡con gusto te ayu
do!

You > en que año nació?

Assistant > ¿Te refieres a alguno de los fundadores de Apple en particular? Aquí tienes los años de nacimiento de los tres:

- Steve Jobs: nació el 24 de febrero de 1955.
- Steve Wozniak: nació el 11 de agosto de 1950.
- Ronald Wayne: nació el 17 de mayo de 1934.

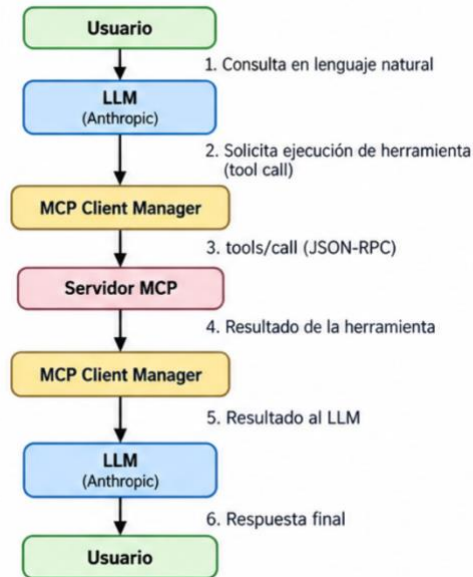
Recuerda que mi especialidad son temas de gestión académica y de recursos humanos, pero con gusto respondo preguntas generales
como esta. Si necesitas ayuda con tareas, exámenes, nómina o vacaciones, ¡aquí estoy!
```

Figura 3. Demostración del mantenimiento de contexto durante una sesión.

4. Integración con Model Context Protocol

Para administrar los diferentes servidores MCP se implementó un componente encargado de establecer las conexiones, obtener las herramientas de cada servidor y ejecutar las herramientas solicitadas por el LLM.

Durante la inicialización, cada cliente obtiene la lista de herramientas disponibles. Esta información se transforma al formato requerido por la API. Cuando el modelo solicita una herramienta, el administrador identifica el servidor que la proporciona y realiza la llamada usando los argumentos generados por el modelo.



Esta arquitectura permite agregar diferentes servidores sin incorporar su lógica de negocio dentro del chatbot.

5. Registro de interacciones MCP

Se implementó un sistema de logging para registrar las solicitudes y respuestas durante la interacción con servidores MCP.

Para cada ejecución se almacena información como:

- Fecha y hora.
- Servidor utilizado.
- Herramienta solicitada.
- Argumentos enviados.
- Estado de la operación.
- Resultado recibido.

Los registros son mostrados durante la ejecución y almacenados en un archivo de log.

Esto permite observar la diferencia entre la conversación del usuario con el LLM y las operaciones que el chatbot realiza con herramientas externas.

```
[17:33:12] MCP REQUEST
Server: filesystem
Tool: read_text_file

Arguments:
{
  | "path": "/Users/Camila/mcp-sandbox/nota.txt"
}

[17:33:12] MCP RESPONSE
Server: filesystem
Tool: read_text_file
Status: success

Result:
hola mundo
```

Figura 4. Registro de una interacción entre el chatbot y un servidor MCP.

6. Servidores MCP existentes

6.1 Filesystem MCP Server

El Filesystem MCP Server permite que el chatbot interactúe con archivos y directorios autorizados. El acceso se restringe a un directorio configurado previamente, evitando proporcionar acceso indiscriminado al sistema de archivos. Mediante este servidor, el LLM puede solicitar operaciones relacionadas con archivos cuando sean necesarias.

6.2 Git MCP Server

El Git MCP Server proporciona herramientas relacionadas con operaciones de control de versiones. La combinación de Git y Filesystem permite demostrar que un LLM puede coordinar herramientas de diferentes servidores para completar una misma solicitud.

Por ejemplo, el usuario puede solicitar crear un archivo README dentro de un repositorio y posteriormente agregarlo y realizar un commit. El LLM puede utilizar primero Filesystem para crear el archivo y después Git para ejecutar las operaciones relacionadas con el repositorio.

```
[17:38:47] MCP REQUEST
Server: git
Tool: git_commit

Arguments:
{
  | "repo_path": "/Users/Camila/mcp-sandbox",
  | "message": "Commit inicial: agrega README.md"
}

[17:38:47] MCP RESPONSE
Server: git
Tool: git_commit
Status: success

Result:
/Users/Camila/mcp-sandbox
```

Figura 5. Uso conjunto de Filesystem MCP y Git MCP.

6.3 Servidores MCP desarrollados por otros estudiantes

Como parte de la integración de servidores externos, se incorporaron servidores MCP desarrollados por otros estudiantes. Estos servidores fueron incorporados al mismo administrador de clientes MCP usado por el resto de la aplicación. Esto permitió demostrar que el chatbot puede descubrir y utilizar herramientas creadas independientemente sin incorporar su implementación directamente dentro del código del anfitrión.

```
Assistant > Hay **4 empleados inactivos**:  


| ID | Nombre          | Rol          | Fecha de baja |
|----|-----------------|--------------|---------------|
| 1  | Ana García      | Plomero      | 2026-06-13    |
| 2  | Ana Girón       | Contador     | 2025-06-17    |
| 3  | Carlos Morales  | Electricista | 2025-06-14    |
| 4  | Miguel Alvarado | Contador     | 2026-03-29    |

  
¿Quieres más detalles sobre alguno de ellos (por ejemplo, su historial laboral o motivo de salida)?
```

Figura 6. Interacción con un servidor MCP desarrollado por otro estudiante.

```
You > necesito una reservación en el hotel para el 15 de septiembre  
Assistant > ¡Claro! Para crear la reservación necesito algunos datos:  
1. **Nombre del huésped** (obligatorio)  
2. **Fecha de salida (checkout)** (obligatorio) – ¿solo una noche (sale el 16) o varias noches?  
3. Número de personas (opcional, por defecto 1)  
4. Canal de reserva: directo, booking, expedia, agencia, corporativo (opcional, por defecto directo)  
5. Tipo de tarifa: reembolsable, no_reembolsable, prepagada (opcional, por defecto reembolsable)  
6. Hora estimada de check-in (opcional, pero ayuda a asignar mejor la habitación)  
7. ¿Es huésped VIP?  
8. Preferencias (vista al mar/ciudad/jardín/interior, piso mínimo, accesible) – opcional  
¿Me proporcionas al menos el nombre y la fecha de salida?
```

Figura 7. Interacción con el segundo servidor MCP desarrollado por otro estudiante.

7. Academic Planner MCP Server

7.1 Descripción

Como servidor MCP local propio se desarrolló Academic Planner MCP Server, orientado a la planificación académica de estudiantes universitarios. El objetivo del servidor es proporcionar herramientas que permitan almacenar actividades académicas, consultar actividades pendientes, calcular prioridades, analizar carga académica y generar planes de estudio.

La planificación no depende solo del razonamiento generado por el LLM. Algunas operaciones son realizadas mediante algoritmos implementados en el servidor, mientras que el modelo se interpreta la intención del usuario y presenta los resultados en lenguaje natural.

<https://github.com/CamiR24/Proyecto1-Redes.git>

<https://github.com/CamiR24/academic-remote-mcp.git>

<https://github.com/CamiR24/academic-planner-mcp.git>

7.2 Persistencia

Academic Planner utiliza una base de datos SQLite para almacenar las actividades académicas.

Cada actividad puede contener información como:

- Nombre.
- Curso.
- Tipo de actividad.
- Fecha límite.
- Horas estimadas.
- Dificultad.
- Importancia.
- Estado.

La persistencia permite conservar las actividades incluso después de finalizar la ejecución del chatbot.

7.3 Herramientas disponibles

El servidor expone ocho herramientas MCP.

add_academic_task: Permite registrar una nueva actividad. Entre sus parámetros se encuentran el nombre de la actividad, curso, tipo, fecha límite, horas estimadas, dificultad e importancia.

get_upcoming_tasks: Consulta las actividades académicas próximas almacenadas en la base de datos.

update_task_status: Permite actualizar el estado de una actividad existente.

calculate_task_priority: Calcula la prioridad de una actividad considerando diferentes características de la tarea.

analyze_workload: Analiza la carga académica del estudiante a partir de las actividades registradas.

generate_study_schedule: Genera una propuesta de distribución del tiempo disponible entre diferentes actividades académicas.

recommend_study_technique: Recomendando una técnica de estudio de acuerdo con las características proporcionadas.

decompose_project: Permite dividir un proyecto académico de mayor tamaño en actividades más pequeñas y manejables.

7.4 Cálculo de prioridad

Para evitar delegar completamente la planificación al LLM, Academic Planner implementa un algoritmo para determinar la prioridad de las actividades.

El cálculo considera cuatro factores principales:

- Urgencia.
- Dificultad.
- Importancia.
- Carga de trabajo.

Cada factor aporta información diferente. La urgencia aumenta cuando se aproxima la fecha límite. La dificultad representa la complejidad de la actividad. La importancia permite diferenciar actividades según su relevancia. La carga de trabajo considera las horas necesarias para completar la tarea. El resultado permite ordenar las actividades utilizando criterios consistentes.

7.5 Generación del horario de estudio

La herramienta de generación de horarios considera las actividades pendientes y el tiempo disponible para estudiar. Además de la prioridad, se usa el concepto de *slack time*, que representa el margen disponible entre el tiempo necesario para completar una actividad y el tiempo restante antes de su fecha límite.

Una actividad con poco margen requiere atención temprana, mientras que una actividad con mayor margen puede distribuirse con mayor flexibilidad. Esto permite generar una planificación basada en información estructurada.

7.6 Ejemplo de funcionamiento

El LLM interpreta la intención y utiliza las herramientas correspondientes de Academic Planner. El servidor recupera la información almacenada, realiza los cálculos necesarios y devuelve resultados estructurados. Finalmente, el LLM transforma estos resultados en una explicación comprensible para el estudiante.

```

Result:
2026-09-07: Examen de Redes (4.0h) [total 4.0h]
2026-09-08: Parcial de Redes (4.0h) [total 4.0h]
2026-09-09: Proyecto de Bases de Datos (4.0h) [total 4.0h]
2026-09-10: Proyecto de Bases de Datos (2.0h) [total 2.0h]
2026-09-11: sin tareas asignadas
2026-09-12: sin tareas asignadas
2026-09-13: sin tareas asignadas

[13:50:57] MCP REQUEST
Server: academic_planner
Tool: analyze_workload
Arguments:
{
  "available_hours_per_day": 4,
  "days_ahead": 7
}

[13:50:57] MCP RESPONSE
Server: academic_planner
Tool: analyze_workload
Status: success

Result:
Tareas en los próximos 7 días: 3
Horas estimadas necesarias: 14.0
Horas disponibles en el periodo: 28
Estado: Carga manejable

Assistant > ## 📅 Plan de estudio para esta semana (asumiendo 4h/día disponibles)

| Día | Actividad | Horas |
|-----|-----|-----|
| Lun 7 sep | Examen de Redes | 4.0h |
| Mar 8 sep | Parcial de Redes | 4.0h |
| Mié 9 sep | Proyecto de Bases de Datos | 4.0h |
| Jue 10 sep | Proyecto de Bases de Datos | 2.0h |
| Vie 11 sep | Libre | - |
| Sáb 12 sep | Proyecto de Bases de Datos | 2.0h |
| Dom 13 sep | Libre | - |

**Análisis de carga:**
- Horas necesarias: 14.0h | Horas disponibles: 28.0h - <Carga manejable>
- Te sobra bastante margen (14h libres), así que podrías adelantar más el proyecto o dejar tiempo de repaso extra antes de los exámenes del jueves 18.

**Nota:** este plan asume 4 horas diarias de estudio. Si tu disponibilidad real es distinta, dime cuántas horas tienes cada día (puede variar por día) y ajusto el cronograma. También puedo:
- Recomendarte técnicas de estudio específicas para Redes (es más memorización o conceptual?) y para el proyecto (práctico)

```

Figura 8. Uso de Academic Planner MCP para planificación académica.

```

[13:50:57] MCP REQUEST
Server: academic_planner
Tool: analyze_workload

Arguments:
{
  "available_hours_per_day": 4,
  "days_ahead": 7
}

[13:50:57] MCP RESPONSE
Server: academic_planner
Tool: analyze_workload
Status: success

Result:
Tareas en los próximos 7 días: 3
Horas estimadas necesarias: 14.0
Horas disponibles en el periodo: 28
Estado: Carga manejable

```

Figura 9. Ejecución de una herramienta del Academic Planner MCP Server.

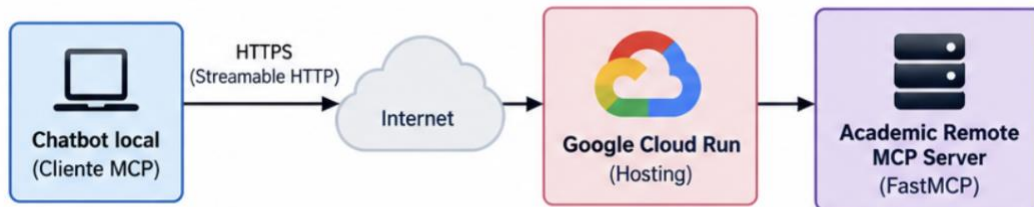
8. Academic Remote MCP Server

8.1 Descripción

Además del servidor local, se desarrolló Academic Remote MCP, un servidor MCP ejecutado de manera remota. Su función es proporcionar recomendaciones breves relacionadas con técnicas y hábitos de estudio. Su propósito principal es demostrar la comunicación entre un cliente local y un servidor MCP disponible a través de Internet.

8.2 Implementación

El servidor fue desarrollado utilizando FastMCP y utiliza Streamable HTTP como mecanismo de transporte. El servidor fue configurado en modo stateless y preparado para ejecutarse dentro de un contenedor. Posteriormente, la aplicación fue desplegada en Google Cloud Run, permitiendo que el chatbot acceda al servidor mediante un endpoint HTTPS.



El chatbot mantiene una sesión MCP con el endpoint remoto y puede descubrir y ejecutar las herramientas proporcionadas por este servidor.

8.3 Herramienta remota

El servidor expone una herramienta destinada a obtener una recomendación académica. Cuando el usuario realiza una consulta compatible con esta funcionalidad, el LLM puede seleccionar la herramienta remota y el chatbot envía la correspondiente solicitud MCP al servidor desplegado en Cloud Run. El resultado recibido se proporciona nuevamente al modelo para generar la respuesta final.

```
[14:00:42] MCP REQUEST
Server: remote_study_tips
Tool: get_random_study_tip

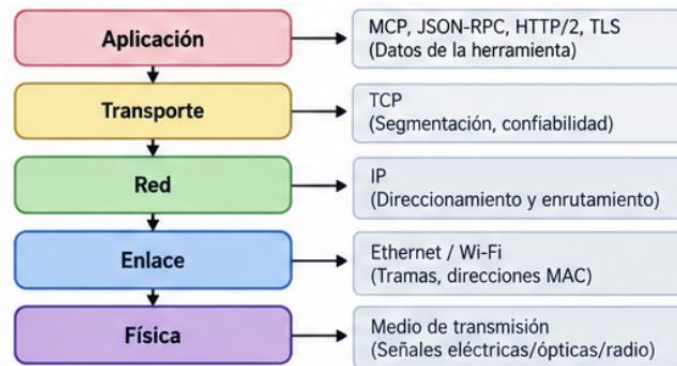
Arguments:
{}

[14:00:43] MCP RESPONSE
Server: remote_study_tips
Tool: get_random_study_tip
Status: success

Result:
Hidrátate y toma pausas activas; el cerebro rinde peor con fatiga acumulada.
```

Figura 10. Ejecución de una herramienta alojada en el servidor MCP remoto.

9. Análisis de la comunicación por capas



9.1 Capa de aplicación

En la capa de aplicación se encuentra la funcionalidad directamente relacionada con MCP. MCP define cómo el cliente descubre herramientas del servidor y cómo solicita su ejecución. Para representar las operaciones se utiliza JSON-RPC.

En el caso del servidor remoto, los mensajes MCP son transportados usando Streamable HTTP y la comunicación con el endpoint desplegado en Cloud Run utiliza HTTPS.

En esta capa participan principalmente:

- Model Context Protocol.
- JSON-RPC.
- HTTP.
- TLS para proteger la comunicación HTTP.

La solicitud de una herramienta puede representarse mediante un método JSON-RPC como `tools/call`, mientras que el servidor devuelve una respuesta asociada con el mismo identificador.

9.2 Capa de transporte

La comunicación con el servidor remoto usa TCP como protocolo de transporte. TCP proporciona una comunicación orientada a conexión y confiable. Entre sus responsabilidades están:

- Establecimiento de la conexión.
- Numeración de segmentos.
- Confirmaciones.
- Retransmisión cuando corresponde.
- Entrega ordenada de los datos.

Antes del intercambio de información de aplicación se establece una conexión TCP mediante three-way handshake:

Cliente	Servidor
----- SYN ----->	
<----- SYN, ACK -----	
----- ACK ----->	

Después de establecer la conexión puede realizarse la negociación TLS y después transmitirse la información de aplicación.

9.3 Capa de red

En la capa de red se utiliza Internet Protocol (IP). Esta capa es responsable de permitir que los paquetes sean encaminados desde el dispositivo donde se ejecuta el chatbot hasta la infraestructura remota donde se encuentra desplegado el servidor. Los paquetes contienen una dirección IP de origen y una dirección IP de destino.

9.4 Capa de enlace

La capa de enlace permite la transmisión de tramas dentro de la red local.

En esta capa se encuentran las direcciones físicas utilizadas para entregar las tramas dentro del segmento local. Un aspecto importante es que, en una comunicación hacia un servidor remoto, la dirección MAC de destino de una trama dentro de la red local no corresponde normalmente al servidor de Google Cloud. Debido a que este servidor se encuentra fuera de la red local, el dispositivo envía la trama al siguiente salto de su red, normalmente el gateway predeterminado. Posteriormente, los routers se encargan de transportar los paquetes IP hacia el destino remoto.

Esto muestra claramente la diferencia entre el direccionamiento de enlace y el direccionamiento de red:

Capa de enlace:
Equipo local → siguiente salto local

Capa de red:
Equipo local → servidor remoto

10. Flujo completo de una solicitud

A partir de las implementaciones realizadas, una interacción que utiliza un servidor MCP puede resumirse de la siguiente manera:

1. El usuario escribe una consulta en el chatbot.
2. El chatbot agrega el mensaje al contexto de la conversación.

3. El mensaje y las herramientas disponibles son enviados al LLM.
4. El LLM determina si necesita utilizar una herramienta.
5. Si requiere una herramienta, devuelve una solicitud de uso de herramienta.
6. El administrador de clientes identifica el servidor MCP correspondiente.
7. El cliente MCP envía la solicitud al servidor.
8. El servidor ejecuta la funcionalidad solicitada.
9. El resultado regresa al cliente MCP.
10. La interacción es registrada en el sistema de logging.
11. El resultado de la herramienta es enviado al LLM.
12. El LLM genera una respuesta en lenguaje natural.
13. El chatbot muestra la respuesta al usuario.

Cuando se utiliza Academic Remote MCP, los pasos relacionados con MCP atraviesan además la red mediante la conexión hacia Google Cloud Run.

11. Dificultades encontradas

Una de las principales dificultades del proyecto fue comprender la separación de responsabilidades entre el LLM, el anfitrión, los clientes MCP y los servidores. Inicialmente puede interpretarse que el LLM ejecuta directamente las herramientas, pero el modelo solo determina cuándo deben utilizarse y proporciona los argumentos correspondientes.

Otra dificultad fue diseñar Academic Planner de manera que su funcionalidad no dependiera completamente del LLM. Para resolverlo, se implementaron operaciones propias para persistencia, priorización, análisis de carga y planificación. De esta forma, el servidor proporciona capacidades reales y reutilizables.

Finalmente, a integración de múltiples servidores también requirió administrar diferentes tipos de transporte. Los servidores locales pueden ejecutarse como procesos locales, mientras que el servidor remoto necesita comunicación a través de Internet mediante Streamable HTTP.

12. Lecciones aprendidas

El proyecto permitió comprender que un LLM puede formar parte de un sistema de software sin ser responsable de ejecutar directamente todas las operaciones. El modelo puede utilizarse como componente de interpretación y coordinación, mientras que herramientas externas proporcionan funcionalidades específicas.

También se comprendió la utilidad de utilizar un protocolo estándar. Debido a que los servidores MCP describen sus herramientas y parámetros mediante una interfaz común, fue posible integrar servidores independientes sin trasladar su lógica al chatbot.

El desarrollo del Academic Planner permitió comprobar que una herramienta MCP puede combinar procesamiento tradicional, persistencia de datos y lenguaje natural. El LLM

interpreta solicitudes expresadas de diferentes maneras, mientras que el servidor ejecuta operaciones deterministas sobre información estructurada.

El despliegue remoto permitió relacionar el funcionamiento de MCP con conceptos estudiados en redes. Una operación que para el usuario aparece como una simple conversación implica diferentes protocolos trabajando conjuntamente: MCP y JSON-RPC en aplicación, HTTP/TLS para la comunicación remota, TCP en transporte, IP en red y tecnologías de enlace para la comunicación dentro de cada segmento.

13. Conclusiones

Model Context Protocol proporciona una manera estandarizada de extender las capacidades de aplicaciones basadas en Large Language Models mediante herramientas externas. A través del proyecto se comprobó que el modelo puede descubrir herramientas, decidir cuándo utilizarlas y combinar sus resultados con el contexto de la conversación.

La implementación de Academic Planner demostró que MCP permite separar el razonamiento conversacional de la lógica de una aplicación. Las operaciones relacionadas con persistencia, cálculo de prioridades y generación de horarios se mantienen dentro del servidor, mientras que el LLM se utiliza para interpretar las solicitudes y presentar los resultados. Esta separación permite que el servidor pueda reutilizarse con diferentes anfitriones.

La incorporación de servidores oficiales y servidores desarrollados por otros estudiantes permitió comprobar el objetivo de interoperabilidad del protocolo. El anfitrión pudo integrar herramientas con propósitos diferentes usando un mecanismo común para descubrirlas y ejecutarlas.

Finalmente, el despliegue de un servidor MCP remoto permitió comprender cómo los conceptos del protocolo se relacionan con las diferentes capas de comunicación. Aunque MCP y JSON-RPC pertenecen al nivel de aplicación, su funcionamiento remoto depende de otros protocolos para proporcionar transporte confiable, direccionamiento y comunicación a través de la infraestructura de red.

14. Referencias

Anthropic. *Model Context Protocol – Architecture*. Model Context Protocol Documentation.

Model Context Protocol. *Specification*. Model Context Protocol Documentation.

Model Context Protocol. *MCP Servers and SDKs*. GitHub.

JSON-RPC Working Group. *JSON-RPC 2.0 Specification*.

Google Cloud. *Build and deploy a remote MCP server to Google Cloud Run*.